



Présentation de la technologie XML

PLAN

- ➡ **Présentation de XML**
- ➡ **La structure des documents XML**
- ➡ **La validation des documents XML: DTD et XMLSchema**
- ➡ **L'extraction de fragments XML: XPath, Xquery**

Présentation de XML

- Le langage XML (*eXtended Markup Language*) est une spécification du W3C depuis février 1998, mais les premiers travaux autour de XML débutent dès 1996.
- XML n'est pas une nouveauté mais une succession d'un ensemble de technologies tels que le SGML, HTML ...
- Pour donner un aperçu rapide de XML, on peut le définir tel un langage (ou une méthode) permettant de structurer des données en employant un format texte comme support

Présentation de XML

- Pourquoi XML ?
 - HTML n'est pas extensible, il ne peut pas répondre aux besoins spécifiques de tous les domaines (mathématiques, chimie, musique, astronomie...) et ne définit plus le contenu du document
 - SGML, qui permettrait de définir de nouveaux langages de balisage spécifiques, est complexe pour le web

Présentation de XML

- Le XML, en lui-même, ne fait rien !
 - Alors que le Html a été conçu pour afficher de l'information, le XML a été créé pour structurer de l'information. Il ne fait rien d'autre !
 - Voici un exemple de XML.

```
<?xml version="1.0"?>
<demoXML>
  <message>Voici du XML
</message>
</demoXML>
```

Ce qui donne avec IE

```
<?xml version="1.0" ?>
- <demoXML>
  <message>Voici du XML</message>
</demoXML>
```

Présentation de XML

□ Quelques applications :

- ✓ **MathML** *Mathematical Markup Language* notation mathématique sur le web
- ✓ **AML** *Astronomical Markup Language* langage décrivant les différents types de données utilisées en astronomie.
- ✓ **CML** *Chemical Markup Language* pour la publication Internet des formules chimiques, de molécules
- ✓ **VML** *Vector Markup Language* langage de balisage d'information graphique vectorielle
- ✓ **SMIL** *Synchronized Multimedia Integration Language* pour la création multimédia, il spécifie comment et quand des éléments multimédia peuvent apparaître dans une page web
- ✓ **WML** *Wireless Markup Language* le langage de balisage pour l'internet mobile

Présentation de XML

```
<?xml version="1.0" encoding="iso-8859-1"?>
<xhtml:html xmlns:xhtml="http://www.w3.org/1999/xhtml">
  <xhtml:body>
    <math display="block"
      xmlns="http://www.w3.org/1998/Math/MathML">
      <msup>
        <mrow>
          <mo>(</mo>
          <mi>a</mi>
          <mo>+</mo>
          <mi>b</mi>
          <mo>)</mo>
        </mrow>
        <mn>2</mn>
      </msup>
    </math>
  </xhtml:body>
</xhtml:html>
```

Ici on décrit l'expression : $(a+b)^2$

Présentation de XML

- XML repose sur le balisage d'un flux de texte, comme HTML.
- La différence fondamentale entre les deux langages est qu'il n'existe aucune limitation quant aux balises de XML !
- En fait, chacun peut inventer ses propres balises.
- Une balise (tag) se symbolise de la façon suivante : **<nom de balise>** : on parle dans ce cas de balises ouvrante.
- Pour fermer une balise, on utilise le format suivant : **</nom de balise>**.

Présentation de XML

- Dans l'exemple précédent `<demoXML>` et `<message>` sont des exemples de balises (tags).
- Les balises sont des chaînes de caractères Unicode encadrées par les caractères « `<` » et « `>` ».
- Entre une balise de début et la balise de fin correspondante, on peut trouver du texte ou d'autres balises.



Présentation de XML

- Pour schématiser, un fichier XML est composé de deux sortes de texte Unicode :
 - **Du texte de balisage.** Les balises sont simplement une technique d'écriture pour représenter la notion conceptuelle d'élément.
 - **Des données textuelles** séparées par des balises. Les données textuelles sont les contenus des éléments.
- Lorsque des éléments sont imbriqués dans d'autres éléments, ils génèrent des structures arborescentes.

Présentation de XML

- Pour comprendre un document XML, il est nécessaire de disposer d'un répertoire des balises utilisées dans ce document, ainsi que d'une **grammaire** décrivant la structuration des balises entre elles.
- Ces documents indispensables à la compréhension métier des fichiers XML sont les DTD (**DocumentType Definition**) ou les **schémas XML**.
- XML est souvent défini comme un « langage de balises sémantiques ».

Principes du langage XML

- Les documents XML sont destinés essentiellement à être traités par des programmes informatiques appelés **processeurs XML**.
- Les documents XML doivent être facilement lisibles par l'homme
- Un document XML doit obligatoirement être **bien formé** : obéir à la spécification XML
 - Chaque élément doit avoir une balise de début et la balise de fin correspondante
 - Les balises doivent être correctement imbriquées entre elles
 - Tout document XML doit avoir un et un seul **élément racine** : un élément qui contient tous les autres
 - Les valeurs associées aux attributs doivent être encadrées par les délimiteurs « “ » ou « ‘ »

Principes du langage XML

- Document XML bien formé (vs valide)
 - Les noms des balises doivent commencer par une lettre ou "_", les autres caractères peuvent être des chiffres, des lettres, "_", "." ou "-".
 - Les noms des balises ne doivent pas commencer par xml
- **ATTENTION:** XML est key sensitive

EXEMPLES

1

```
<?xml version="1.0" encoding="ISO-8859-1"?>
```

```
<personne>
```

```
  <nom>
```

```
  <prenom>
```

```
  </nom>
```

```
  </prenom>
```

```
<personne>
```

Ko : Mauvaise imbrication

5

```
<?xml version="1.0" encoding="ISO-8859-1"?>
```

```
<personne>
```

Ko : Balise non fermée

```
<?xml version="1.0" encoding="ISO-8859-1"?>
```

```
<personne/>
```

Ok : Élément vide

2

```
<?xml version="1.0" encoding="ISO-8859-1"?>
```

```
<personne>
```

```
</Personne>
```

Ko : Personne différent de personne

4

3

```
<?xml version="1.0" encoding="ISO-8859-1"?>
```

```
<xlmime>
```

```
</xlmime>
```

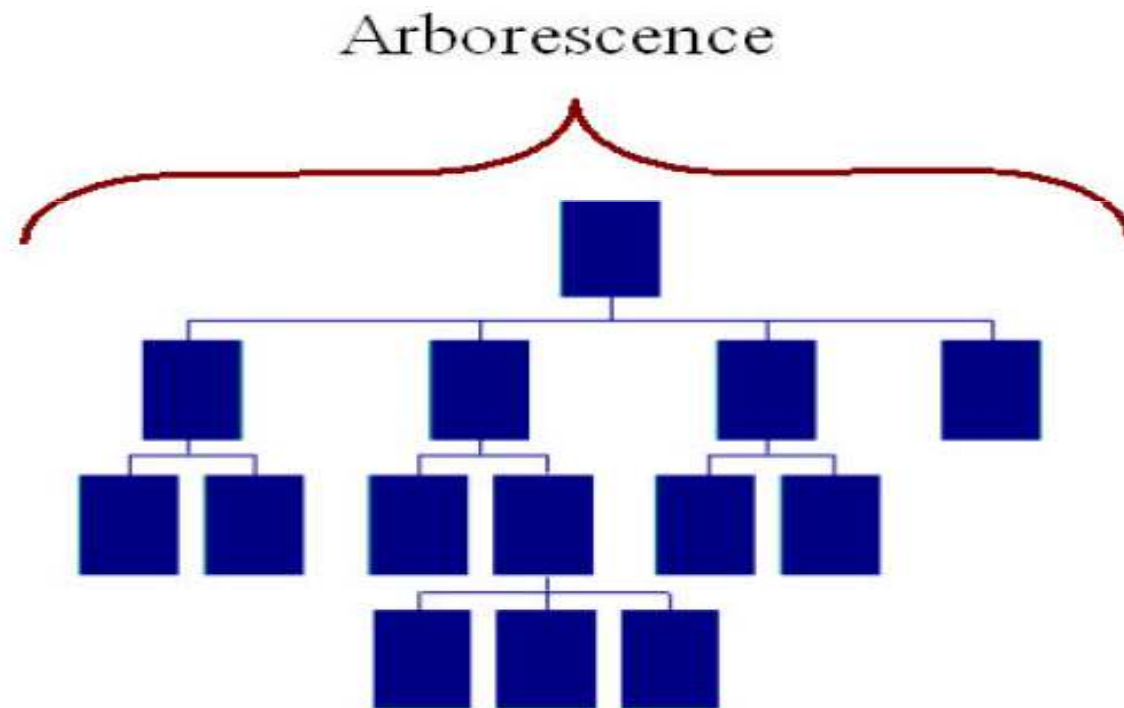
Ko : Pas de nom de balise commençant par xml



Documents XML : Structure Logique et Structure Physique

Structure des documents XML

- La structure logique d'un document XML est une arborescence: structure hiérarchique et ordonnée



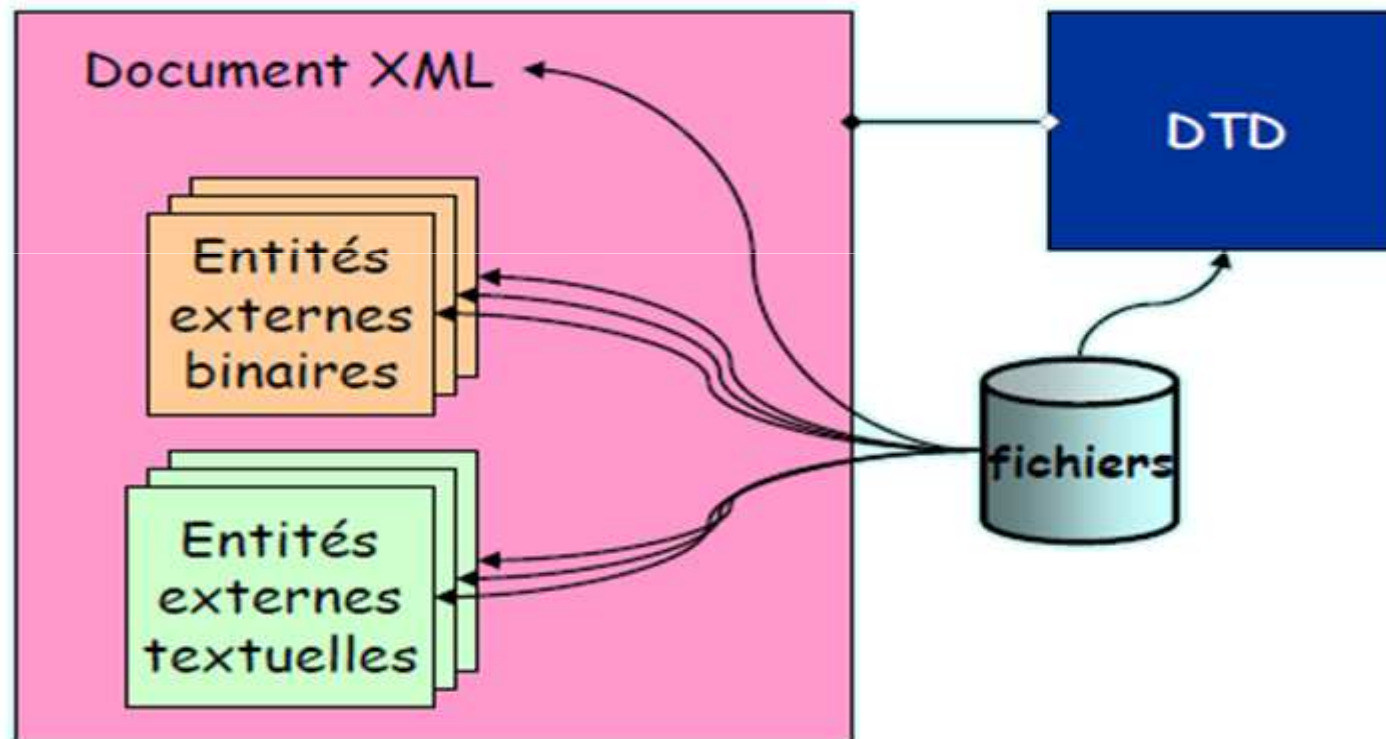
Structure des documents XML

- La structure logique d'un document XML est une arborescence d'informations obtenue par un processeur XML à partir d'un contenu physique constitué d'une simple suite linéaire de caractères Unicode.
- Du point de vue logique, un document XML est composé :
 - D'éléments constituant la grammaire d'organisation ;
 - De textes constituant les données ;
 - éventuellement
 - De déclarations contenues dans un préambule,
 - De commentaires,
 - D'instructions de traitement.

Structure des documents XML

- Un document XML peut exister physiquement d'un seul bloc mais dépend le plus souvent de l'assemblage de plusieurs unités physiques existant séparément et appelées entités.
- Les entités peuvent être schématisées comme des fichiers provenant de différents moyens de stockage situés à différents emplacements
- Chaque entité peut elle-même appeler d'autres entités

Structure des documents XML



Structure des documents XML

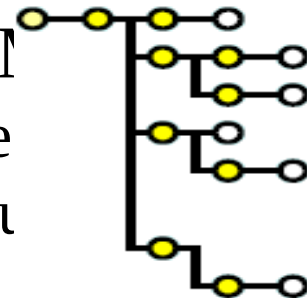
❑ La structure physique d'un document XML

- ❖ Séquence de caractères "à plat"
- ❖ Spécifie l'encodage caractère
- ❖ Spécifie le découpage en fichiers
- ❖ Peut contenir des caractères non significatifs (indentations)

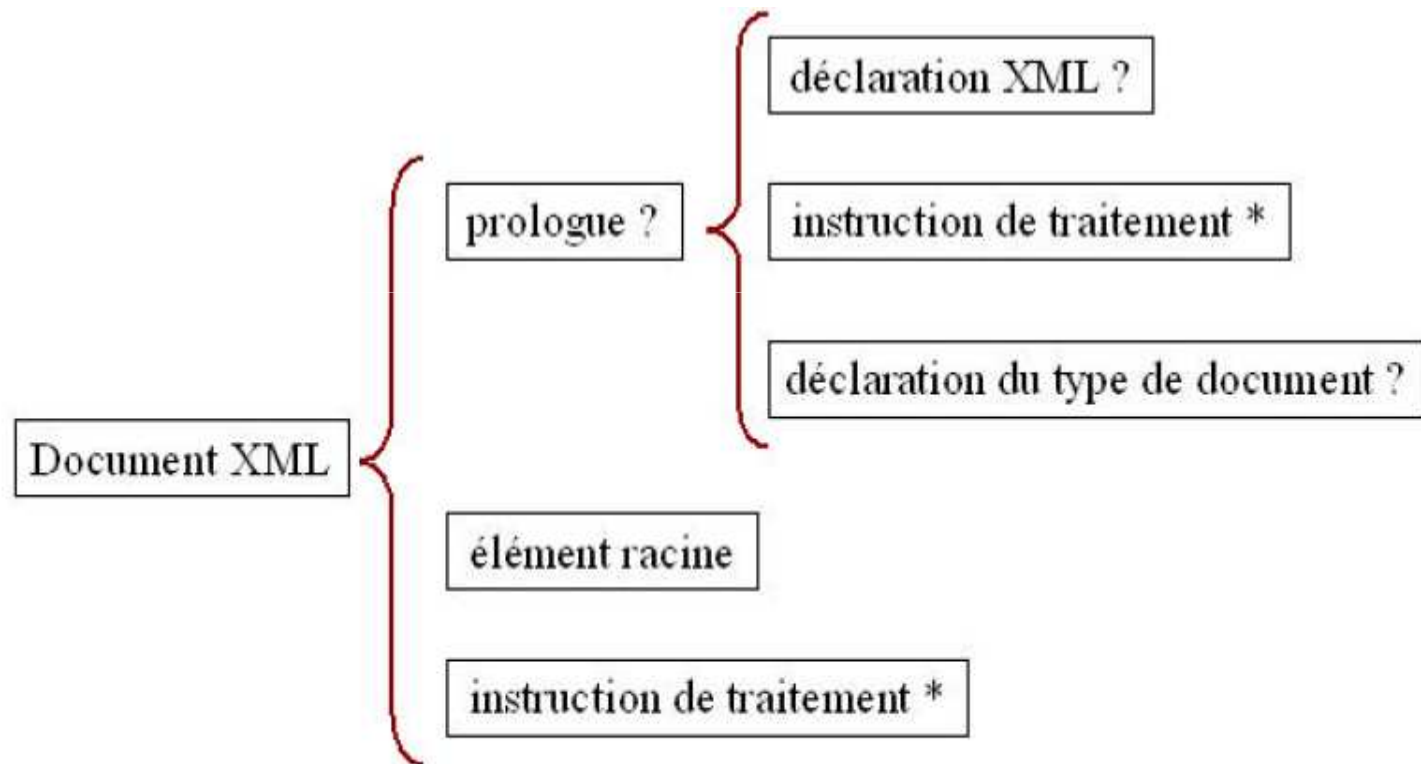
```
<?xml version="1.0" ?>  
- <demoXML>  
  <message>Voici du XML</message>  
</demoXML>
```

❑ La structure logique d'un document XML

- ❖ Une arborescence d'informations obtenue par le processeur XML à partir du contenu physique



Structure des documents XML



Préambule et déclarations

- une déclaration XML
- `<?xml version='1.0' encoding="ISO-8859-1" ?>`
- Le **préambule** (prolog) s'il est présent, précède le texte de balisage et les données textuelles d'un document. Il contient des déclarations qui peuvent être les suivantes :
 - La **déclaration XML**. Elle n'est pas obligatoire. La spécification XML précise cependant qu'elle devrait être placée au début d'un document XML. Si elle est présente, elle doit apparaître en premier

Préambule et déclarations

- une déclaration de type de document

<!DOCTYPE bulletin_météo SYSTEM "meteo.dtd" []>

- **La déclaration de type de document** (Document Type Declaration : différente de la DTD). Elle n'est obligatoire que pour la validation. Si elle est présente, elle doit se trouver dans le préambule et suivre la déclaration XML. La déclaration de type de document permet de référencer et/ou définir localement deux sortes d'entités :
 - Des définitions de type de document (DTD) contenant des déclarations de balisage ;
 - Des entités (autres que des DTD).

Préambule et déclarations

- La déclaration XML débute par « **<?xml** » et se termine par « **?>** ». Elle fournit trois informations sous la forme de pseudo-attributs.
- **version** : cet attribut est obligatoire quand la déclaration XML est présente **<?xml version="1.0" ?>**
- **encoding** : cet attribut indique le type de codage que doivent utiliser les processeurs XML afin de traduire des octets en caractères Unicode à la réception d'un document, ou de transformer des caractères Unicode en données binaires pour le transport.
- Il permet à des applications de se comprendre.
- **<?xml version="1.0" encoding="ISO-8859-1" ?>**

Préambule et déclarations

- **standalone** : cet attribut indique la nature autonome ou non d'un document.
- Un document est déclaré non-autonome lorsque l'entité document qui le représente est dépendante de déclarations de balisage externes

`<?xml version="1.0" encoding="UTF8" standalone="no" ?>`

Structure des documents XML

❑ La déclaration XML : encoding

Norme	Correspondance
UTF-8	Jeu de caractères universel sur 8 bits
UTF-16	Jeu de caractères universel sur 16 bits
Norme	Correspondance
ISO-8859-1	Latin 1 – Langues d'Europe de l'ouest et d'Amérique latine
ISO-8859-2	Latin 2 – Langues d'Europe centrale et Slaves
ISO-8859-3	Latin 3 – Langues Espéranto, Galicienne, Maltaise et Turc
ISO-8859-4	Latin 4 – Langues Estonienne, Lettonne et Lithuanienne
ISO-8859-5	Langue Cyrilliques
ISO-8859-6	Langue Arabe
ISO-8859-7	Langue Grecque
ISO-8859-8	Langue Hébraïque
ISO-8859-9	Latin 5 – Langue Turc
ISO-8859-10	Latin 6 – Langues Groenlandaises et Lapones

Commentaires et instructions de traitement

- Les commentaires peuvent être placés dans un fichier XML ou un fichier DTD :
- `<!-- commentaire -->`
- Instructions de traitement (Processing Instructions)
 - Instructions spéciales pour les applications qui manipulent les documents XML : `<?cible arg1 arg2 ...?>`
 - cible : nom de l'application
 - arg1, arg2 : arguments passées à l'application
 - `<?xml-stylesheet type="text/css" href="style.css" ?>`
 - Cette instruction indique au navigateur d'afficher les données XML en appliquant la feuille de style : "style.css"

Structure physique : Entités

- Un document XML est physiquement découpé en entités
- Qu'est-ce qu'une entité ?
 - Une entité est un fragment nommé de document
 - Ces sont des « raccourcis » vers des portions de documents allant du caractère au document complet
- Ces entités doivent être définies dans l'en-tête du document XML, ou dans la DTD, et peuvent être référencées à une ou plusieurs reprises dans le document.



Structure physique : Entités

- Pourquoi utiliser des entités ?
 - Pour gagner du temps dans les mises à jour
 - Pour mémoriser des caractères particuliers ou des phrases fréquentes

Entité XML définie dans le document

- Une telle entité sert en général d'abréviation pour des informations répétitives ou qui peuvent être modifiées en bloc:

```
<?xml version="1.0"?>  
<!DOCTYPE démo [  
  <!ELEMENT démo (#PCDATA|important)>  
  <!ELEMENT important (#PCDATA)>  
  <!ENTITY cie "Les aliments de Dakar SA">  
  <!ENTITY imp "<important>Attention!</important>">  
<démo>  
  &cie;  
  &imp;  
</démo>
```
- Ce document déclare les entités **cie** et **imp**, et les utilise. Pour ce faire, on utilise une balise formée du caractère "&", suivi du nom de l'entité, suivi du caractère ";".

Entité XML définie à l'externe

- **Entité XML définie à l'externe**
- Les entités définies à l'extérieur des documents sont utilisées de la même façon mais sont déclarées ainsi :

```
<?xml version="1.0"?>  
<!DOCTYPE doc [  
    <!ENTITY chap1 SYSTEM "../chapitres/chap1.xml">  
    <!ENTITY chap2 SYSTEM "../chapitres/chap2.xml">  
]>  
<doc>  
    &chap1;  
    &chap2;  
</doc>
```
- Dans cet exemple, on suppose que les fichiers "chap1.xml" et "chap2.xml" contiennent les chapitres marqués en XML.

Types d'entités

- On distingue :
 - entités prédéfinies
 - entités caractères
 - entités générales qui sont des fragments de l'élément du document,
 - entités paramètres qui sont des fragments de DTD.

Déclaration d'une entité

- **Déclaration d'une entité paramètre :**
 - **interne**
 - `<!ENTITY% nom "entité">`
 - **externe**
 - `<!ENTITY% nom SYSTEM nom du fichier contenant l'entité>`

Déclaration d'une entité

- **Déclaration d'une entité générale :**
 - **interne**
 - `<!ENTITY nom "entité">`
 - **externe**
 - `<!ENTITY nom SYSTEM "nom du fichier contenant l'entité">`

Déclaration d'une entité

- **Par exemple :**
- `<!ENTITY % identificateur ID #REQUIRED>`
- `<!ENTITY ref "refuge">`
- `<!ENTITY monentite SYSTEM
"mon_site/entites.xml">`

Référence à une entité

- Une référence à une entité paramètre a la forme suivante :

%nom;

- Par exemple :
- `<!ATTLIST renvoi cible %identificateur>`
- au lieu de :
- `<!ATTLIST renvoi cible ID #REQUIRED>`

Référence à une entité

- Une référence à une entité générale a la forme suivante :

&nom;

- Par exemple :
- `<para>S'élever au-dessus du &ref; des Drayères en direction est. Suivre...</para>`
- au lieu de :
- `<para>S'élever au-dessus du refuge des Drayères en direction est. Suivre...</para>`

Entités caractères

- Un caractère non disponible sur la station de travail peut être représenté par son code Unicode en décimal ou en hexadécimal, sous la forme d'une référence à une entité :
 - *&#code décimal;*
 - *ode hexadécimal;*
- **Par exemple :**
 - *&* caractère & (son code ascii est 38 et hexadécimal 26)
 - *Φ* lettre grecque Φ
 - © s'écrit *©* ou *©*

Entités prédéfinies

- Les caractères < > & ' " qui sont des délimiteurs XML peuvent être remplacés dans un texte par une référence à une entité prédéfinie.
- Ces entités sont les suivantes :
 - < réfère le caractère <
 - > réfère le caractère >
 - & réfère le caractère &
 - ' réfère le caractère '
 - " réfère le caractère "

Espaces de noms

- L'importation d'éléments ou d'attributs contenus dans des entités externes peut entraîner des conflits de noms
- Ces conflits peuvent être évités en définissant des espaces de noms
- Les espaces de noms en XML, c'est un moyen
 - d'éviter les conflits de noms.
 - d'identifier de façon universelle les noms utilisés dans les documents



Espaces de noms

- Les documents XML sont eux-mêmes des espaces de noms :
 - Les noms d'éléments doivent être uniques dans un document ;
 - De plus, chaque élément est un espace de noms pour ses attributs

Déclaration d'un espace de noms

- La déclaration d'un espace de noms et de son préfixe associé consiste à insérer dans la balise ouvrante d'un élément contenant des noms (d'éléments ou d'attributs) issus de cet espace, l'attribut :
- `xmlns:préfixe="URI de l'espace de noms"`
- On peut déclarer un espace de noms par défaut par l'attribut :
- `xmlns="URI de l'espace de noms"`

Exemple d'espace de noms

```
<?xml version="1.0" encoding="UTF-8" ?>
<dossier_scolaire>
  <dénomination xmlns="http://uadb.edu.sn/etudiants">
    Abdou Gaye
  </dénomination>
  <dénomination xmlns="http://uadb.edu.sn/diplomes">
    Mastère
  </dénomination>
  <dénomination xmlns="http://uadb.edu.sn/matieres">
    Informatique
  </dénomination>
</dossier_scolaire>
```

Exemple d'espace de noms

- Supposons que
 - l'URI « monSite/dtdLivre.xml » contienne une DTD pour la description de livres et
 - que les éléments auteur, editeur et annee du guide « Itinéraires skieurs » soient conformes à ces définitions, la description de ce guide pourrait être la suivante :

```
<guide xmlns:livre="monSite/dtdLivre.xml">
<livre:titre>Itinéraires skieurs dans la Vallée
de la Clarée </livre:titre>
<livre:auteur>Jean-Gabriel
Ravary</livre:auteur>
<livre:editeur>Le Polygraphe</livre:editeur>
<livre:annee>1991</livre:annee>
...
<vallou>
<nom>Vallon des Muandes</nom>
...
</guide>
```



La validation des documents

XML: DTD

Structure physique : DTD

- Une DTD (Document Type Definition) est une sorte d'entité contenant une description formelle de la structure et du vocabulaire (noms de type d'éléments et noms d'attributs) d'un document.
- La spécification XML définit complètement l'écriture des DTD,
- Mais une DTD n'est pas elle-même un document XML

Structure physique : DTD

- Un processeur validateur lit la DTD avant de lire le document.
- Une DTD
 - Permet de définir le "vocabulaire" et la structure qui seront utilisés dans le document XML
 - Grammaire du langage dont les phrases sont des documents XML (instances)
 - Peut être mise dans un fichier et être appelée dans le document XML

Structure physique : DTD

- La DTD utilise la déclaration DOCTYPE qui référence des déclarations soit internes ou externes
- **DTD interne :**
 - Les déclarations de la DTD sont placés dans le DOCTYPE après la déclaration XML
 - L'attribut standalone a la valeur "yes" `<?xml version="1.0" standalone="yes">`
 - `<!DOCTYPE document [déclarations]> <document > ... </document >`
 - **document** : nom de la balise racine du document déclaré,
 - **déclarations** : déclarations d'éléments, d'attributs et d'entités...

Structure physique : DTD

- **DTD externe** : permet d'inclure des documents XML identifiés par une URI
- `<!DOCTYPE document SYSTEM "uri">`
- Les déclarations sont placés dans un fichier séparé
- L'attribut standalone a la valeur "no"
- `<!DOCTYPE document SYSTEM
"http://uadb.edu.sn/document.dtd">`

Structure physique : DTD

- `<!DOCTYPE document (PUBLIC "identifiant_public" "url"?)>`
- PUBLIC est utilisé lorsque la DTD est une norme ou qu'elle est enregistrée sous forme de norme ISO par l'auteur

Structure physique : DTD

- Déclaration de listes d'attributs
 - `<!ATTLIST Nom_élément Nom_attribut type_attribut défaut>`
 - `<!ATTLIST document nd (1|2|3) "1">`
- Type d'attributs :
 - CDATA : chaîne de caractères prise telle quelle
 - ID ou IDREF : renvoi dans le document
 - ENTITY ou ENTITIES : entité externe non XML
 - NMTOKEN (s): noms symboliques formés de caractères alphanumériques sans espace (plusieurs NMTOKEN)

Structure physique : DTD

- Déclaration d'un élément :
- `<!ELEMENT balise valeur>`
- Décrit une balise **balise** qui fera partie du vocabulaire
- ordre spécifié :
- `<!ELEMENT balise (val1, val2, val3)>`
- Choix exclusif :
- `<!ELEMENT balise (val1 | val2 | val3)>`
- Indicateur d'occurrence d'un élément
- Contenu textuel déclaré comme PCDATA (parseable character data)

Structure physique : DTD

- Opérateurs d'occurrence

- **Notations**

- (a, b) séquence
- (a | b) liste de choix
- a? élément optionnel [0,1]
- a* élément répétitif [0,N]
- a+ élément répétitif [1,N]

Exemples

(nom, prenom, rue, ville)

(oui | non)

(nom, prenom?, rue, ville)

(produit*, client)

(produit*, vendeur+)

Structure physique : DTD

- Valeur par défaut des attributs:
 - **#REQUIRED** : attribut obligatoirement présent et valeur spécifié par le créateur de l'instance
 - **#IMPLIED** : présence de l'attribut facultative
 - **#FIXED** "Valeur" : l'attribut prend toujours cette valeur

Structure physique : DTD

- Données dans un élément : simple
 - `<!ELEMENT Donnees (#PCDATA)>`
 - les données sont constituées par un flot de caractères
- Modèle mixte : complexe
 - `<!ELEMENT mixte (#PCDATA | val1 | val2)*>`
 - dans cet exemple l'élément mixte est composé de données et d'éléments

Quelques exemples

- Contenu libre :
 - `<!ELEMENT libre ANY>`
- Élément vide :
 - `<!ELEMENT vide EMPTY>`
- Attributs pour un élément :
 - `<!ATTLIST balise [attribut type #mode [valeur]]>`
- Définition de la liste des attributs pour une balise
 - `<!ATTLIST enseignant nom CDATA #REQUIRED
pre nom CDATA #IMPLIED>`
 - `<!ATTLIST adresse ville CDATA #FIXED "Dakar">`

Quelques exemples

- Liens hypertextuels internes : ID ou IDREF
- Dans la DTD
 - `<!ELEMENT SECTION (#PCDATA|xref)*>`
 - `<!ATTLIST SECTION TARGET ID #IMPLIED>`
 - `<!ELEMENT xref EMPTY>`
 - `<!ATTLIST xref ref IDREF #REQUIRED>`
- Dans l'instance
 - `<SECTION TARGET="cible"> contenu
</SECTION>`
 - `<SECTION>` référence à la section
 - `<xref ref="cible"/>`
 - `</SECTION>`

Ecrire un document XML valide destiné à transmettre l'énoncé suivant:

- <!DOCTYPE textemath [
- <!ELEMENT textemath ((texte | formule)+) >
- <!ELEMENT texte (#PCDATA) >
- <!ELEMENT formule
(valeur|somme|difference|produit|fraction|racine|puissance) >
- <!ELEMENT valeur (#PCDATA)>
- <!ELEMENT somme (op1, op2)>
- <!ELEMENT difference (op1, op2)>
- <!ELEMENT produit (op1, op2)>
- <!ELEMENT fraction (op1, op2)>
- <!ELEMENT racine (op1)>
- <!ATTLIST racine ordre CDATA #IMPLIED>
- <!ELEMENT puissance (op1)
- <!ATTLIST puissance exposant CDATA #REQUIRED>
- <!ELEMENT op1 (valeur|formule)>
- <!ELEMENT op2 (valeur|formule)>]>

Calculer la valeur de l'expression $\frac{x^4 + \sqrt[3]{5}}{7\sqrt{3}}$ lorsque x prend la valeur 4 .